Clément PITRAT

Institut Polytechnique des Sciences Avancées

Final Assignment Report

Non-Preemptive Schedule Search

Student: Clément PITRAT 4TS1

Professor: Jasdeep Singh

Course: Real-Time Scheduling

Submission Date: 28 April 2026

Academic Year 2025–2026

Contents

1 Introduction **2**

2 Assignment Requirements **2**

3 Execution Time Measurement of Task τ_1 **3**

4 Assumptions **3**

5 Final Task Set **3**

6 Hyperperiod and Job Generation **4**

7 Schedulability Analysis **4**

7.1 Processor Utilization 4

7.2 Total Execution Time and Idle Time 5

8 Scheduling Algorithm **5**

9 Normal Non-Preemptive EDF Schedule **5**

10 Second Schedule: τ_5 Allowed to Miss **6**

11 Comparison of the Two Schedules **7**

12 Computational Complexity **7**

13 Conclusion **8**

Git Repository **8**

A Appendix A: Source Code Structure **9**

B Appendix B: Main Scheduling Logic **9**

C Appendix C: Response Time Verification **9**

D Appendix D: Notes on the Second Schedule **9**

1 Introduction

The objective of this assignment is to measure the execution time of task τ_1, determine its worst-case execution time C_1, and use it in a periodic real-time task set.

The final goal is to generate a non-preemptive job-level schedule over one hyperperiod. The schedule must verify that no deadlines are missed, and the response time of each job must be checked against its deadline.

A second schedule is also studied where task τ_5 is allowed to miss a deadline. In this implementation, this second case is modeled by penalizing τ_5 in the priority function.

The implementation is divided into two parts:

- a C program used to measure the execution time of τ_1;
- a Python scheduler used to generate jobs and build the schedules.

2 Assignment Requirements

The assignment requires the following tasks:

- measure the execution time of τ_1;
- obtain the minimum, maximum, Q1, Q2, Q3 and WCET of τ_1;
- use the measured value to determine C_1;
- check schedulability of the task set;
- generate a non-preemptive schedule at job level;
- verify that no deadlines are missed;
- compute the total waiting time;
- discuss processor idle time;
- generate a second schedule where τ_5 is allowed to miss a deadline;
- provide the computational complexity of the algorithm;
- list each job response time and verify it against its deadline.

Task	Computation time C_i	Period T_i
τ_1	C_1	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

Table 1: Initial task set.

3 Execution Time Measurement of Task τ_1

Task τ_1 was implemented in C. Its execution time was measured over 200 runs. The obtained measurements are expressed in milliseconds.

Statistic	Execution time (ms)
Minimum	0.643
Q1	0.707
Median / Q2	0.758
Q3	0.843
Measured WCET / Maximum	1.278

Table 2: Measured execution time statistics for task τ_1.

The measured WCET is:

$$C_{1,\text{measured}} = 1.278 \text{ ms}$$

For the schedulability analysis, a conservative value is used by rounding the measured WCET up to the next integer:

$$C_1 = \lceil 1.278 \rceil = 2$$

Therefore, the value used in the task set is $C_1 = 2$. This value is conservative because it is greater than the maximum measured execution time.

4 Assumptions

The following assumptions were made:

- Tasks are periodic.
- Relative deadlines are equal to periods, therefore $D_i = T_i$.
- The scheduler is non-preemptive.
- All jobs are generated over one hyperperiod.
- The measured WCET of τ_1 is rounded up to obtain a conservative value.
- The scheduling policy used is non-preemptive Earliest Deadline First.
- In the second schedule, task τ_5 is penalized in the priority function because it is allowed to miss a deadline.
- All execution times and periods are treated in the same time unit.

5 Final Task Set

After measuring and rounding the WCET of τ_1, the final task set becomes:

Task	Computation time C_i	Period / Deadline $T_i = D_i$
τ_1	2	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

Table 3: Final task set used by the scheduler.

6 Hyperperiod and Job Generation

The hyperperiod is the least common multiple of all periods:

$$H = \text{lcm}(10, 10, 20, 20, 40, 40, 80) = 80$$

The schedule is therefore constructed over the interval $[0, 80]$.
The number of jobs generated for each task is:

$$N_i = \frac{H}{T_i}$$

Task	Period T_i	Number of jobs	Total execution time
τ_1	10	8	$8 \times 2 = 16$
τ_2	10	8	$8 \times 3 = 24$
τ_3	20	4	$4 \times 2 = 8$
τ_4	20	4	$4 \times 2 = 8$
τ_5	40	2	$2 \times 2 = 4$
τ_6	40	2	$2 \times 2 = 4$
τ_7	80	1	$1 \times 3 = 3$

Table 4: Jobs generated over one hyperperiod.

The total number of jobs is:

$$J = 8 + 8 + 4 + 4 + 2 + 2 + 1 = 29$$

7 Schedulability Analysis

7.1 Processor Utilization

The utilization is computed as:

$$U = \sum_i \frac{C_i}{T_i}$$

Using the final task set:

$$U = \frac{2}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} = 0.8375$$

Since $U < 1$, the processor demand is lower than the available processing capacity. This indicates that the task set is feasible from a utilization point of view.

7.2 Total Execution Time and Idle Time

The total execution time over one hyperperiod is:

$$C_{\text{total}} = 16 + 24 + 8 + 8 + 4 + 4 + 3 = 67$$

The total idle time is:

$$I = H - C_{\text{total}} = 80 - 67 = 13$$

Therefore, 13 time units are idle over the hyperperiod.

8 Scheduling Algorithm

The scheduling algorithm used is non-preemptive Earliest Deadline First.

At each scheduling decision:

1. all released jobs are identified;
2. among the available jobs, the job with the earliest absolute deadline is selected;
3. the selected job is executed until completion;
4. the current time is updated to the finish time of the selected job.

For the second schedule, the same method is used, but task τ_5 is penalized:

$$P_j = \begin{cases} d_j + 1000, & \text{if } j \in \tau_5, \\ d_j, & \text{otherwise.} \end{cases}$$

where d_j is the absolute deadline of job j.

9 Normal Non-Preemptive EDF Schedule

Table 5 shows the job-level schedule obtained using normal non-preemptive EDF.

Table 5: Normal non-preemptive EDF schedule.

Job	Interval	Deadline	Waiting	Response	Status
$T1_1$	[0,2]	10	0	2	OK
$T2_1$	[2,5]	10	2	5	OK
$T3_1$	[5,7]	20	5	7	OK
$T4_1$	[7,9]	20	7	9	OK
$T5_1$	[9,11]	40	9	11	OK
$T1_2$	[11,13]	20	1	3	OK
$T2_2$	[13,16]	20	3	6	OK
$T6_1$	[16,18]	40	16	18	OK
$T7_1$	[18,21]	80	18	21	OK
$T1_3$	[21,23]	30	1	3	OK

Job	Interval	Deadline	Waiting	Response	Status
$T2_3$	[23,26]	30	3	6	OK
$T3_2$	[26,28]	40	6	8	OK
$T4_2$	[28,30]	40	8	10	OK
$T1_4$	[30,32]	40	0	2	OK
$T2_4$	[32,35]	40	2	5	OK
$T1_5$	[40,42]	50	0	2	OK
$T2_5$	[42,45]	50	2	5	OK
$T3_3$	[45,47]	60	5	7	OK
$T4_3$	[47,49]	60	7	9	OK
$T5_2$	[49,51]	80	9	11	OK
$T1_6$	[51,53]	60	1	3	OK
$T2_6$	[53,56]	60	3	6	OK
$T6_2$	[56,58]	80	16	18	OK
$T1_7$	[60,62]	70	0	2	OK
$T2_7$	[62,65]	70	2	5	OK
$T3_4$	[65,67]	80	5	7	OK
$T4_4$	[67,69]	80	7	9	OK
$T1_8$	[70,72]	80	0	2	OK
$T2_8$	[72,75]	80	2	5	OK

The total waiting time is:

$$W_{\text{total}} = 140$$

The number of missed deadlines is 0. All jobs verify:

$$R_j \leq d_j$$

where R_j is the response time and d_j is the absolute deadline.

10 Second Schedule: τ_5 Allowed to Miss

In the second schedule, τ_5 is penalized in the priority function. This means that, when τ_5 is available, it may be delayed behind other jobs.

Table 6: EDF schedule with τ_5 penalized.

Job	Interval	Deadline	Waiting	Response	Status
$T1_1$	[0,2]	10	0	2	OK
$T2_1$	[2,5]	10	2	5	OK
$T3_1$	[5,7]	20	5	7	OK
$T4_1$	[7,9]	20	7	9	OK
$T6_1$	[9,11]	40	9	11	OK
$T1_2$	[11,13]	20	1	3	OK
$T2_2$	[13,16]	20	3	6	OK
$T7_1$	[16,19]	80	16	19	OK
$T5_1$	[19,21]	40	19	21	OK
$T1_3$	[21,23]	30	1	3	OK

Job	Interval	Deadline	Waiting	Response	Status
$T2_3$	[23,26]	30	3	6	OK
$T3_2$	[26,28]	40	6	8	OK
$T4_2$	[28,30]	40	8	10	OK
$T1_4$	[30,32]	40	0	2	OK
$T2_4$	[32,35]	40	2	5	OK
$T1_5$	[40,42]	50	0	2	OK
$T2_5$	[42,45]	50	2	5	OK
$T3_3$	[45,47]	60	5	7	OK
$T4_3$	[47,49]	60	7	9	OK
$T6_2$	[49,51]	80	9	11	OK
$T1_6$	[51,53]	60	1	3	OK
$T2_6$	[53,56]	60	3	6	OK
$T5_2$	[56,58]	80	16	18	OK
$T1_7$	[60,62]	70	0	2	OK
$T2_7$	[62,65]	70	2	5	OK
$T3_4$	[65,67]	80	5	7	OK
$T4_4$	[67,69]	80	7	9	OK
$T1_8$	[70,72]	80	0	2	OK
$T2_8$	[72,75]	80	2	5	OK

The total waiting time is:

$$W_{\text{total}} = 141$$

The number of missed deadlines is 0.

Although task τ_5 was allowed to miss a deadline, no deadline miss occurred in this particular case. This shows that, for the measured and rounded value $C_1 = 2$, the task set still has enough slack to schedule all jobs before their deadlines.

11 Comparison of the Two Schedules

Schedule	Total waiting time	Deadline misses
Normal non-preemptive EDF	140	0
EDF with τ_5 penalized	141	0

Table 7: Comparison of the two schedules.

The normal EDF schedule gives a lower total waiting time than the second schedule. Therefore, among the two tested schedules, the normal non-preemptive EDF schedule is selected as the best one.

12 Computational Complexity

Let J be the number of jobs generated over one hyperperiod.

At each scheduling step, the algorithm scans the remaining jobs to find the available jobs and then selects the one with the earliest deadline.

There are J scheduling decisions, and each decision requires scanning up to J jobs. Therefore, the worst-case complexity is:

$$O(J^2)$$

The number of jobs is:

$$J = \sum_{i=1}^{n} \frac{H}{T_i}$$

Therefore, the complexity can also be expressed as:

$$O\left(\left(\sum_{i=1}^{n} \frac{H}{T_i}\right)^2\right)$$

For this assignment, $J = 29$, so the algorithm is computationally inexpensive for this task set.

13 Conclusion

The execution time of task τ_1 was measured experimentally over 200 runs. The measured WCET was:

$$1.278 \text{ ms}$$

A conservative rounded value was used for schedulability analysis:

$$C_1 = 2$$

Using this value, the processor utilization is:

$$U = 0.8375 < 1$$

The task set is feasible from a utilization point of view. A non-preemptive EDF schedule was generated over the hyperperiod $H = 80$. All jobs met their deadlines, and the total waiting time was:

$$W_{\text{total}} = 140$$

A second schedule was also generated where τ_5 was penalized and allowed to miss a deadline. However, no deadline miss occurred, and the total waiting time increased to:

$$W_{\text{total}} = 141$$

Therefore, the normal non-preemptive EDF schedule is the best schedule among the two implemented cases.

Git Repository

The source code is available at:

`https://github.com/CPITRAT/real-time-final-assignment.git`

A Appendix A: Source Code Structure

The project contains two main files:

- `task1.c`: measures the execution time of task τ_1;
- `scheduler.py`: computes the statistics of τ_1, generates jobs, and builds the schedules.

B Appendix B: Main Scheduling Logic

The main scheduling rule is summarized below:

```
available = [j for j in jobs if j["release"] <= time]

if penalize_t5:
    job = min(
        available,
        key=lambda j: j["deadline"] + (1000 if j["task"] == 5 else 0)
    )
else:
    job = min(available, key=lambda j: j["deadline"])
```

In the normal schedule, the selected job is the available job with the earliest absolute deadline.

In the second schedule, task τ_5 receives a deadline penalty. This makes it less likely to be selected before other available jobs.

C Appendix C: Response Time Verification

For each job, the response time is computed as:

$$R_j = f_j - r_j$$

where:

- f_j is the finish time of job j;
- r_j is the release time of job j.

A job meets its deadline if:

$$R_j \leq D_j$$

or equivalently:

$$f_j \leq d_j$$

where d_j is the absolute deadline of the job. In the normal non-preemptive EDF schedule, all jobs satisfy this condition.

D Appendix D: Notes on the Second Schedule

The assignment allows task τ_5 to miss a deadline in the second schedule. In this implementation, τ_5 was penalized in the scheduling priority.

However, with the measured and rounded value $C_1 = 2$, the task set still has enough slack. Therefore, even after penalizing τ_5, no deadline miss occurred.

This result is not contradictory: it means that allowing τ_5 to miss does not necessarily imply that a miss must occur for this specific task set.